

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский авиационный институт  
(Национальный исследовательский университет)»

Факультет информационных технологий и прикладной математики  
Кафедра вычислительной математики и программирования

ОТЧЁТ  
по лабораторной работе № 3  
по дисциплине «Операционные системы»

Студент: Князьков Николай Дмитриевич

Группа: М80–203БВ–24

Вариант: 3

Преподаватель: Соколов А. А.

Москва, 2025

## Лабораторная работа № 3

### 1.1 Постановка задачи

Цель работы: освоение принципов работы с файловыми системами и обмена данными через технологию «memory-mapped files».

Задание: повторить задачу лабораторной работы №1 (вариант 3: деление первого введённого числа на последующие) с тем отличием, что взаимодействие между процессами организуется через отображаемые файлы (memory-mapped files) вместо pipe. Основной (родительский) процесс порождает дочерний, и обмен данными происходит через специально созданный файл, отображённый в память.

### 1.2 Общие сведения о программе

Программа состоит из двух частей: родительский процесс и дочерний процесс. Вместо неименованного канала используется файл, открытый и отображённый в память функцией `mmap()`. Родительский процесс создаёт файл фиксированного размера или с необходимой длиной (с `open()` и `ftruncate()`) и отображает его в память. Затем создаётся дочерний процесс (`fork()`). Оба процесса получают доступ к одной и той же области памяти. Родитель записывает числа в разделяемый участок памяти. Дочерний процесс читает из этой области, выполняет деление и записывает результаты обратно. По завершении работы вызываются `munmap()` и `close()`.

Системные вызовы: `open()`, `ftruncate()`, `mmap()`, `munmap()`, `fork()`, `waitpid()`, `close()`. Хэдеры: `<sys/mman.h>`, `<fcntl.h>`, `<sys/stat.h>`, `<unistd.h>`, `<sys/wait.h>`.

### 1.3 Метод и алгоритм решения

Родительский процесс создаёт и открывает файл, выделяет в нём память под массив чисел и отображает его с помощью `mmap()`. После `fork()` дочерний процесс видит ту же область памяти. Родитель записывает числа, дочерний выполняет деления и записывает результаты. Если обнару-

жен нулевой делитель, дочерний записывает код ошибки и завершает оба процесса. Родитель ожидает завершения дочернего (`waitpid()`). После выполнения оба процесса освобождают память и закрывают файл (`munmap()` и `close()`).

Алгоритм делений совпадает с ЛР1, за исключением использования отображаемой памяти вместо pipe.

## 1.4 Основные файлы программы с кодом

Файл `lab3.c` демонстрирует использование `mmap`:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <sys/wait.h>
6 #include <unistd.h>
7
8 int main() {
9     const char *filename = "shared.dat";
10    int fd = open(filename, O_RDWR | O_CREAT, 0666);
11    ftruncate(fd, 4096); //
12    int *data = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
MAP_SHARED, fd, 0);
13    if (data == MAP_FAILED) perror("mmap");
14    pid_t pid = fork();
15    if (pid == 0) {
16        //
17        //                                     data[0..n-1],
18
19                                     data
20        munmap(data, 4096);
21        close(fd);
22        exit(0);
23    } else {
24        //
25        //
26        data,
        waitpid(pid, NULL, 0);
        munmap(data, 4096);
        close(fd);
```

```
27     }  
28     return 0;  
29 }
```

Листинг 1.1: lab3.c

## 1.5 Пример работы

После запуска `./lab3` родительского модуля можно ввести «100 4 5». Эти числа окажутся в разделяемой памяти, дочерний процесс делит 100 на 4 и записывает 25, затем 25 на 5 и получает 5. Файл `shared.dat` будет содержать результат. При делении на 0 выводится ошибка, оба процесса завершают работу.

## 1.6 Выводы

В лабораторной работе показано использование отображаемых файлов для межпроцессного обмена. Результаты идентичны ЛР1: первое число делится на последующие, остановка при делении на ноль. Применён механизм `mmap()` и `munmap()`, что упрощает обмен данными. Основные преимущества: совместный доступ к большой области памяти. Задача реализована корректно, продемонстрированы навыки работы с memory-mapped files и системными вызовами (`open`, `mmap`, `fork`, `close`).